

KeepClip — dokumentacja infrastruktury

Narzędzia zewnętrzne, integracja z Windows, Google Drive, bufor powtórek, budowanie i wydania, testy, diagnostyka, skalowanie

Wersja aplikacji 2.3.0 · gałąź `csharp` · stan na 17 września 2026

Ten dokument opisuje wszystko wokół kodu aplikacji: z jakich zewnętrznych składników KeepClip korzysta, jak łączy się z systemem Windows i z Google, jak działa nagrywanie powtórek, jak buduje się instalator i publikuje wydanie, jak działają testy i jak rozwijać projekt dalej. Endpointy i bazę danych opisuje dokument *Backend*, a interfejs dokument *Frontend*.

Spis treści

1. Składniki zewnętrzne
2. Integracja z Windows
 - 2.1 Okno (DesktopShell / ShellForm)
 - 2.2 Globalny skrót (HotkeyManager)
 - 2.3 Powiadomienie o powtórcie (ReplayToast)
 - 2.4 Kosz (Trash)
 - 2.5 Menedżer poświadczeń (CredentialStore)
 - 2.6 Sprzęt (GpuInfo, DisplayHelper)
3. Google Drive: OAuth i transfer
 - 3.1 Klient OAuth
 - 3.2 Przepływ łączenia (PKCE)
 - 3.3 Tokeny i pamięć podręczna
 - 3.4 Operacje na plikach
 - 3.5 Prywatność
4. Bufor natychmiastowej powtórki (ReplayService)
 - 4.1 Potok
 - 4.2 Przechwytywanie ekranu — trzy poziomy
 - 4.3 Kodowanie
 - 4.4 Audio (AudioPump)
 - 4.5 Nadzór
 - 4.6 Zapis (SaveAsync)
5. Transkrypcja: środowisko Whispera
6. Budowanie i wydanie
 - 6.1 Konfiguracje
 - 6.2 Instalator (Inno Setup)
 - 6.3 Wersjonowanie i CI
7. Testy automatyczne
 - 7.1 Uruchamianie
 - 7.2 Izolacja od danych użytkownika
 - 7.3 Rodzaje testów
 - 7.4 Jak działa uruchomienie aplikacji w teście
 - 7.5 Jak dodać test
 - 7.6 Czego testy nie obejmują i jak to sprawdzać ręcznie
8. Diagnostyka
 - 8.1 Typowe problemy
9. Skalowanie i kierunki rozwoju
 - 9.1 Większe biblioteki (tysiące klipów)
 - 9.2 Więcej funkcji bez chaosu
 - 9.3 Wiele urządzeń / dostęp z sieci
 - 9.4 Jakość i utrzymanie

1. Składniki zewnętrzne

Składnik	Skąd	Gdzie leży	Do czego
Microsoft Edge WebView2 Runtime	Zwykle jest w Windows 10/11; instalator ma bootstrapper (<code>installer/redist/MicrosoftEdgeWebView2Setup.exe</code> , pobierany przez <code>build.ps1</code>) i uruchamia go tylko, gdy w rejestrze <code>EdgeUpdate\Clients\{F3017226-...}</code> nie ma wersji.	System	Okno aplikacji. Brak lub uszkodzenie: okno pokazuje komunikat z odnośnikiem do pobrania i aplikacja się zamyka; szczegóły w <code>data/crash.log</code> .
ffmpeg / ffprobe	<code>setup.ps1</code> pobiera build „release-essentials” z <code>gyan.dev</code> ; w wydaniu kopiuje je cel <code>StageRuntimeAssets</code> w <code>csproj</code> .	<code>tools/bin/</code> (poza gitem)	Miniatury, pomiar czasu, wycinanie, naprawa, kopie podglądowe, dekodowanie audio dla Whispera, przechwytywanie i łączenie powtórek.
Whisper.net 1.9.1 + Runtime + Runtime.Vulkan	NuGet	Biblioteki natywne w <code>runtimes/win-x64</code> i <code>runtimes/vulkan/win-x64</code>	Rozpoznawanie mowy. Whisper.net wybiera Vulkan, gdy GPU jest dostępny, inaczej CPU. Modele GGML pobiera sam do <code>models/</code> .
NAudio 2.2.1	NuGet	w procesie	WASAPI loopback (dźwięk gry) i mikrofon dla powtórek; lista urządzeń.
Microsoft.Data.Sqlite + SQLitePCLRaw.bundle_e_sqlite3 2.1.12	NuGet	<code>e_sqlite3.dll</code>	Baza. Bezpośrednia referencja do bundle wymusza załataną wersję (podatność w 2.1.11).
Google Drive API v3, OAuth 2.0	HTTPS	—	Chmura. Bez SDK Google; własne żądania <code>HttpClient</code> .
GitHub Releases API	HTTPS	—	Sprawdzanie aktualizacji i pobieranie instalatora.
Inno Setup 6	<code>winget / jrsoftware.org</code>	maszyna budująca	Instalator <code>KeepClip-Setup.exe</code> .

Podpisy cyfrowe. Żaden plik w wydaniu nie jest podpisany (`exe`, `KeepClip.dll`, biblioteki Whispera, `ffmpeg`, instalator). Windows 11 z włączonym Smart App Control blokuje niepodpisane biblioteki bez reputacji — po każdym wydaniu użytkownicy zgłaszają blokadę `ggml-whisper.dll` (transkrypcja nie działa). Jedyna trwała naprawa to podpisywanie wszystkich plików w CI (Azure Trusted Signing albo certyfikat, np. Certum). Do tego czasu to znany problem.

2. Integracja z Windows

2.1 Okno (DesktopShell / ShellForm)

- **Położenie okna** zapisywane w ustawieniach jako `max;x;y;w;h` przy zamykaniu; przywracane tylko, gdy prostokąt przecina podłączony ekran (odłączony monitor nie chowa okna poza ekranem). Maksymalizacja odbywa się na ekranie, na którym okno było ostatnio. Domyślnie 1280×800 na środku, minimum 900×600.
- **Zasobnik**: przy włączonym nagrywaniu w tle zamknięcie okna chowa je do zasobnika (ikona, menu „Otwórz / Zakończ i zatrzymaj nagrywanie”, teksty odświeżane w bieżącym języku tuż przed pokazaniem). Zamknięcie przez system lub „Zakończ” naprawdę kończy proces.
- **Pojedyncza instancja**: muteks `Local\KeepClip-SingleInstance`; drugi start sygnalizuje zdarzenie `Local\KeepClip-ShowRequest`, na które czeka wątek `KeepClip-ShowSignal` i przywraca okno z zasobnika.
- **Profil WebView2** leży w `data/webview2`, żeby deinstalator usunął go z aplikacją. Argument przeglądarki `--enable-features=PlatformHEVCDecoderSupport` włącza dekodowanie HEVC. Skrót przeglądarki (F10 i inne) są wyłączone. Nowe okna (np. zgoda Google) otwierają się w przeglądarce systemowej.
- **Wyjątki** na wątku okna trafiają do `crash.log` zamiast systemowego okna błędów (`UnhandledExceptionMode.CatchException`).

2.2 Globalny skrót (HotkeyManager)

`RegisterHotKey` na uchwycie okna z flagą `MOD_NOREPEAT`; rejestracja musi odbyć się w wątku UI. Skrót działa też, gdy gra jest na pełnym ekranie. Wymagany jest co najmniej jeden modyfikator. Gdy inna aplikacja ma ten sam skrót, rejestracja się nie udaje i interfejs pokazuje ostrzeżenie (`hotkey_active=false`). W trybie samego serwera nie ma okna, więc i skrót.

2.3 Powiadomienie o powtórce (ReplayToast)

Małe okno bez aktywacji (`WS_EX_NOACTIVATE` | `WS_EX_TOOLWINDOW` | `WS_EX_TRANSPARENT`) w prawym górnym rogu: ikona ✓/X, tytuł, podtytuł, kolorowy pasek. Pojawia się przez animację przezroczystości i znika tą samą drogą; **nigdy nie jest ukrywane ani zamykane**, bo zmiana stanu okna potrafi zminimalizować grę pełnoekranową. Dźwięk (`frontend/sounds/replay-saved.wav` / `replay-failed.wav`) gra natychmiast po skrócie, bo nakładka nie zawsze jest widoczna nad grą.

2.4 Kosz (Trash)

`SHFileOperation` z `FOF_ALLOWUNDO` | `FOF_NOCONFIRMATION` | `FOF_SILENT` | `FOF_NOERRORUI`. `SendWithRetry` ponawia 6 razy co 250 ms, bo tuż po zamknięciu odtwarzacza plik bywa jeszcze otwarty.

2.5 Menedżer poświadczeń (CredentialStore)

Token odświeżania Google jest zapisywany przez `CredWriteW` jako poświadczenie ogólne `KeepClip:GoogleDriveRefreshToken` z trwałością `LOCAL_MACHINE` (przeżywa wylogowanie i restart tego użytkownika). Limit 2560 bajtów. Nigdy nie trafia do `data/` ani do repozytorium. Deinstalator usuwa go poleceniem `cmdkey /delete`.

2.6 Sprzęt (GpuInfo, DisplayHelper)

- `GpuInfo.LargestDedicatedVideoMemory()` pyta DXGI (`CreateDXGIFactory1` → `EnumAdapters1` → `GetDesc1`) o największą pamięć karty z pominięciem adapterów programowych. WMI zwraca 32-bitowy `AdapterRAM` ucięty do 4 GB, dlatego DXGI. Brak DXGI → 0 → najostrożniejszy model.
- `DisplayHelper.MeasurePrimary()` czyta fizyczną rozdzielczość ekranu głównego przez `EnumDisplaySettings` (niezależnie od skalowania DPI), ogranicza szerokość kodowania do 3840 px (limit NVENC H.264) i wymusza parzyste wymiary.
- `PreferGdiCapture()` zwraca prawdę w Windows Sandbox (`WDAGUtilityAccount`), sesji zdalnej (`TerminalServerSession`, `SESSIONNAME=RDP-...`) albo przy `KEEPCLIP_CAPTURE_GDI=1` — tam Desktop Duplication zakłóca kursor.
- `ForegroundGameName()` bierze nazwę procesu okna na pierwszym planie (pomija Eksploratora i KeepClip), mapuje znane nazwy (`cs2` → Counter-Strike 2), próbuje `ProductName` / `FileDescription` z zasobów pliku i czyści przez `SafeFolderName` (niedozwolone znaki → `_`, obcięcie do 80 znaków, nazwy zastrzeżone `CON`, `AUX`... dostają przedrostek `_`, pusta nazwa → „Pulpit”).

3. Google Drive: OAuth i transfer

3.1 Klient OAuth

Aplikacja używa **jednego wspólnego klienta OAuth autora** (typ „Desktop app”) zapisanego w `data/google_client.json`. Plik nie jest w repozytorium: instalator wbudowuje go z `installer/embed/google_client.json`, a CI odtwarza z sekretu `GOOGLE_CLIENT_JSON_B64` (base64 pliku JSON) i przerywa wydanie, gdy sekretu brak albo nie zawiera `installed_client_id` i `client_secret`. Bez pliku widok Chmura pokazuje instrukcję założenia własnego klienta w Google Cloud Console. Zakres uprawnień to wyłącznie `https://www.googleapis.com/auth/drive.file` — aplikacja widzi tylko pliki, które sama utworzyła.

3.2 Przepływ łączenia (PKCE)

1. POST `/api/cloud/connect`: losowy `code_verifier` (64 bajty), `code_challenge` = SHA-256 w base64url, losowy `state`, lokalny `HttpListener` na `http://127.0.0.1:{wolny port}/`, limit 5 minut. Poprzedni niedokończony przepływ jest anulowany.
2. Adres zgody: `accounts.google.com/o/oauth2/v2/auth` z `access_type=offline` (prosi o token odświeżania) i `prompt=consent` (wymusza jego wydanie także przy ponownej zgodzie).
3. Google przekierowuje na lokalny adres z `code` i `state`. Żądania bez kodu (np. favicon) dostają 204 i nie kończą oczekiwania. Zgodność `state` jest sprawdzana.
4. Wymiana kodu na tokeny (`oauth2.googleapis.com/token`) z `code_verifier`; brak tokenu odświeżania to błąd. Adres e-mail konta trafia do opisu wpisu w Menedżerze poświadczeń.
5. Strona w przeglądarce dostaje prosty HTML „Połączono z Google Drive”. Interfejs KeepClip wykrywa połączenie odpypywaniem statusu.

3.3 Tokeny i pamięć podręczna

Token dostępu żyje w pamięci z zapasem 60 s przed wygaśnięciem; odświeżanie chroni semafor (`TokenGate`). Nowy token odświeżania z odpowiedzi Google nadpisuje zapisany. Błąd `invalid_grant` (cofnięta zgoda, wygasły token) rozłącza lokalnie i wymaga ponownego połączenia. Dane konta (`drive/v3/about`: nazwa, e-mail, zdjęcie, limit, użycie) są trzymane 20 s, a operacje zmieniające zużycie unieważniają je. Rozłączenie usuwa token lokalnie i odwołuje go w Google (błąd odwołania jest ignorowany).

3.4 Operacje na plikach

Operacja	Jak
Folder KeepClip	<code>EnsureFolderAsync</code> : sprawdza zapamiętany identyfikator (<code>cloud_folder_id</code> , także czy nie jest w koszu), szuka po nazwie, w ostateczności tworzy.
Wysyłka	Sesja wznawialna (<code>uploadType=resumable</code>): najpierw metadane (<code>name</code> , <code>parents</code> , <code>appProperties</code>), potem jeden PUT ze strumieniem pliku bez buforowania w pamięci; osobny <code>HttpClient</code> bez limitu czasu dla wielogigabajtowych plików (klient API ma 60 s).
Metadane	<code>appProperties</code> : <code>keepclip="1"</code> , <code>game</code> , <code>duration</code> , <code>mtime</code> — pozwalają odtworzyć bibliotekę na innym komputerze bez bazy.
Lista	<code>files?q='{folder}' in parents and trashed=false</code> , strony po 1000 z <code>pageToken</code> .
Pobranie	<code>files/{id}?alt=media</code> do pliku <code>.part</code> , potem podmiana.
Odtwarzanie z chmury	To samo żądanie z nagłówkiem <code>Range</code> przekazany od przeglądarki; odpowiedź 206 i <code>Content-Range</code> są przepuszczane, więc przewijanie działa bez pobierania całości.
Usunięcie	PATCH <code>files/{id}</code> z <code>{"trashed":true}</code> — kosz Google, nie trwałe usunięcie.

3.5 Prywatność

KeepClip nie ma własnego serwera. Dane idą wyłącznie między komputerem użytkownika a jego Dyskiem Google. Aplikacja nie wysyła telemetrii; zdarzenia samouczka trafiają tylko do lokalnego logu deweloperskiego i tylko jako identyfikatory kroków.

4. Bufor natychmiastowej powtórki (ReplayService)

4.1 Potok

```
AudioPump (NAudio) —PCM f32le→ stdin |
      |
      | ffmpeg → data/tmp/replay/seg_000123.ts (2 s każdy)
ekran (ddgrab / gdigrab) —————▶   ▲ janitor kasuje starsze niż długość powtórki + 2
skrót / przycisk → SaveAsync → ffmpeg concat (-c:v copy, audio AAC) → <biblioteka>/<gra>/<gra> data.mp4
```

4.2 Przechwytywanie ekranu — trzy poziomy

Poziom	Filtr ffmpeg	Kiedy
ddgrab-gpu (0)	ddgrab=framerate=N:draw_mouse=0,null[v] — klatki zostają na GPU	Najszybszy; obecnie start odbywa się od poziomu 1, poziom 0 istnieje w kodzie jako opcja.
ddgrab (1)	ddgrab...,hwdownload,format=bgra,scale=W:H:force_original_aspect_ratio=decrease,pad=W:H:(ow-iw)/2:(oh-ih)/2,format=nv12[v]	Domyślny na zwykłym pulpicie (transfer przez CPU).
gdigrab (2)	-f gdigrab -framerate N -draw_mouse 0 -video_size WxH -i desktop + ta sama normalizacja	Windows Sandbox, sesje zdalne, <code>KEEPCLIP_CAPTURE_GDI=1</code> , albo po dwóch szybkich awariach wyższego poziomu.

Stały, parzysty rozmiar ekranu głównego (skalowanie i wypełnienie) chroni enkoder przed zmianą trybu przez grę. Cursor nie jest dodawany (`draw_mouse=0`), bo kopiowanie sprzętowego kursora powodowało jego migotanie na pulpicie; kursory rysowane przez gry są widoczne.

Znaczniki czasu nie są normalizowane: `ddgrab` stempluje klatki zegarem rzeczywistym, co utrzymuje synchronizację z audio przy zgubionych klatkach. Wcześniejsza próba `setpts=N/fps` zerowała znaczniki (filtr liczy w jednostkach osi czasu, nie w sekundach) i bufor wychodził pusty.

4.3 Kodowanie

Enkoder jest wybierany raz na proces przez krótki test kodowania 3 klatek (`ProbeEncoder`): `h264_nvenc` → `h264_amf` → `h264_qsv` → `libx264`. Stały kwantyzator (jakość: low 28, medium 24, high 20; `NVENC constqp`, `AMF cqp`, `QSV global_quality`, `x264 crf`) trzyma jakość szybkiego ruchu lepiej niż ograniczona przepływność. `GOP = FPS`. Wyjście: `-f segment -segment_time 2 -reset_timestamps 1 -segment_format mpegts` z numeracją kontynuowaną po restarcie (`NextSegNumber`), żeby nie tracić bufora.

4.4 Audio (AudioPump)

Bez jawnego wyboru urządzenia miksuje **wszystkie aktywne wyjścia** (gra nie zawsze gra na domyślnym) plus opcjonalny mikrofon, każde przez `BufferedWaveProvider` (5 s, nadmiar odrzucany), z konwersją częstotliwości i kanałów. Osobny wątek pompuje próbki według zegara ściennego z opóźnieniem 200 ms — dzięki temu strumień nie zatrzymuje się podczas ciszy, a krótkie skoki WASAPI są pokrywane prawdziwymi próbkami. Po 30 s rozjazdu następuje twarda resynchronizacja (np. po uśpieniu). Co 15 s loguje szczyt sygnału, co pozwala wykryć niemy strumień. Audio używa czasu próbek od zera; znaczniki zegara rzeczywistego miały inną skalę niż `ddgrab` i `ffmpeg` przestawał czytać potok. Brak urządzeń audio nie blokuje nagrywania obrazu.

4.5 Nadzór

- Watchdog** czeka na koniec procesu `ffmpeg`. Nieoczekiwany koniec: zatrzymanie, licznik szybkich awarii (poniżej 10 s pracy), po 2 → zejście poziomem przechwytywania, po 6 → zatrzymanie z komunikatem; restart po 700 ms (pierwszy) albo $1,5 \times$ liczba awarii, z zachowaniem segmentów. Cała logika jest w `try/catch` — wyjątek zatrzymuje tylko bufor i trafia do `crash.log`.
- Janitor** co 5 s kasuje segmenty starsze niż $\text{długość}/2 + 2$, chyba że trwa zapis.
- Ostatnie 20 wierszy `stderr` `ffmpeg` jest trzymane do komunikatów błędów.
- `ApplyConfig` porównuje podpis ustawień (`długość|fps|jakość|mikrofon|wyjście|wejście`) i restartuje bufor tylko przy zmianie.

4.6 Zapis (SaveAsync)

- Czeka do 4 s, jeśli bufor właśnie się restartuje; blokuje równoległy zapis (`_saving`).
- Nazwa gry z okna na pierwszym planie; dźwięk i powiadomienie „Zapisuję...” natychmiast.

3. Bierze ukończone segmenty potrzebne na długość powtórki oraz kopię aktualnie pisanego segmentu (jeśli ma ponad 50 kB), żeby nie stracić ostatnich sekund.
4. Plik listy dla demuksera `concat` (ukośniki, apostrofy w zapisie `'\''`); łączenie `-c:v copy` (dla HEVC `-tag:v hvc1`), audio `aac 192k` z `aresample=async=1:first_pts=0,apad` i `-shortest`, limit 120 s.
5. Plik tymczasowy `.part` leży w folderze docelowym (przemianowanie na tym samym woluminie); przeniesienie ponawiane do 5 razy co 400 ms (antywirus, indeksator).
6. Powiadomienie z nazwą pliku, skan biblioteki, miniatury.

5. Transkrypcja: środowisko Whispera

- Whisper.net ładuje bibliotekę natywną w kolejności: Vulkan (dowolne GPU) → CPU. Użyty silnik trafia do dziennika transkrypcji (`Silnik STT: Vulkan (GPU) [model: large-v3]`).
- Modele: `ggml-large-v3.bin` (ok. 3 GB), `ggml-large-v3-turbo.bin` (ok. 1,6 GB), `ggml-small.bin` (ok. 0,5 GB) w `models/` obok aplikacji; pobierane przy pierwszym użyciu przez plik `.part`.
- Dobór: pierwszy model z drabinki, którego próg VRAM jest spełniony (6 GB / 3 GB / 0). Brak pamięci GPU może zabić proces bez wyjątku — znacznik `whisper_model_probe` w ustawieniach wykrywa to przy następnym starcie i blokuje model (`whisper_model_blocked`). Wyjątek w trakcie transkrypcji schodzi na lżejszy model w tej samej sesji.
- Zmienna `KEEPCLIP_WHISPER_MODEL` wymusza model i wyłącza automatykę (przydatne w diagnostyce).
- Deinstalator usuwa `models/`; aktualizacja w miejscu go nie rusza.

6. Budowanie i wydanie

6.1 Konfiguracje

Polecenie	Efekt
<code>dotnet build KeepClip/KeepClip.csproj -c Release</code>	Build lokalny z <code>KEEPCLIP_DEV</code> (panel logów, <code>/api/logs</code>). Aplikacja musi być zamknięta (<code>taskkill /IM KeepClip.exe /F</code>), inaczej błąd MSB3021.
<code>dotnet run --project KeepClip</code>	Uruchomienie z repozytorium; <code>data/</code> i <code>models/</code> są wspólne dla wszystkich konfiguracji (odnajdywane w górę drzewa).
<code>installer\build.ps1 [-PublicRelease] [-SkipPublish] [-NoInstaller]</code>	Publikacja samowystarczalna win-x64 (bez przycinania i bez pojedynczego pliku, bo WebView2 i Whisper muszą być osobnymi plikami), usunięcie bloków DEV z interfejsu w trybie publicznym, przycięcie nieużywanych runtime'ów (x86, arm64, linux, osx), Inno Setup → <code>installer/dist/KeepClip-Setup.exe</code> .

6.2 Instalator (Inno Setup)

- Instalacja dla użytkownika w `%LOCALAPPDATA%\KeepClip` bez uprawnień administratora; stały `AppId` i `CloseApplications=yes` dają aktualizację w miejscu — działający KeepClip (także w zasobniku) jest zamykany przez Menedżera ponownego uruchamiania, a biblioteka, transkrypcje, ustawienia i połączenie z Google zostają.
- Bootstrapper WebView2 uruchamiany tylko przy braku składnika; `google_client.json` kopiowany do `{app}\data`.
- Deinstalacja: usuwa token Google (`cmdkey /delete`), `data/` i `models/`, ale nigdy klipów użytkownika.
- Języki instalatora: polski i angielski.

6.3 Wersjonowanie i CI

- Numer wersji w **dwóch** miejscach: `KeepClip.csproj` (`Version`, `FileVersion`, `AssemblyVersion`) i `installer/KeepClip.iss` (`MyAppVersion`). Muszą się zgadzać; commit „Release X.Y.Z”.
- Tag `vX.Y.Z` uruchamia `.github/workflows/release.yml` (windows-latest): checkout, .NET 10, **kontrola tłumaczeń i testy xUnit**, pobranie ffmpeg (`setup.ps1`), odtworzenie klienta OAuth z sekretu, `build.ps1 -PublicRelease`, `gh release create` z instalatorem i automatycznymi notatkami.
- Ponowne uruchomienie workflow dla istniejącego tagu kończy się błędem na kroku tworzenia wydania (wydanie już istnieje) — to nie jest awaria builda.
- `.github/workflows/ci.yml` uruchamia te same testy przy każdym pushu i pull requestcie na `csharp` (z pominięciem zmian tylko na stronie WWW i w plikach Markdown).
- Strona `keepclip.pl` (`keepclip-web/`) publikuje się osobnym workflow `pages.yml` na GitHub Pages przy zmianach w tym katalogu.
- Aplikacja sama sprawdza wydania (6 h pamięci podręcznej) i instaluje je w miejscu; więcej w dokumencie Backend przy `/api/update/*`.

Wydanie jest nieodwracalne i publiczne. Kolejność: zmiany → build → własne testy → sprawdzenie w aplikacji → dopiero potem podbicie wersji, tag i push tagu.

7. Testy automatyczne

7.1 Uruchamianie

```
dotnet test tests/KeepClip.Tests/KeepClip.Tests.csproj -c Release
node tests/i18n-check.js
```

Projekt `tests/KeepClip.Tests` (xUnit 2.9, `Microsoft.AspNetCore.Mvc.Testing`, `net10.0-windows` z WinForms) referuje projekt aplikacji. Klasy wewnętrzne są widoczne dzięki `InternalsVisibleTo`. Cały przebieg trwa kilka sekund; 124 testy.

7.2 Izolacja od danych użytkownika

Inicjalizator modułu (`TestEnvironment`) ustawia przed pierwszym użyciem klasy `Config`: `KEEPCLIP_DATA_DIR` na katalog w `%TEMP%\KeepClip.Tests\<guid>`, `KEEPCLIP_SERVER_ONLY=1` (bez muteksu i okna) i `KEEPCLIP_IDLE_EXIT=0`. Po zakończeniu procesu katalog jest usuwany. Testy nie wymagają ffmpeg, modelu Whispera ani konta Google: sprawdzają, że bez tych narzędzi aplikacja zwraca czysty błąd, a nie awarię. Równoległość klas jest wyłączona, bo aplikacja trzyma ścieżki w polach statycznych; każdy test zaczyna od `TestDb.Reset()` (czyszczenie tabel) i/lub `TestSettings.Reset()` (usunięcie `settings.json`).

7.3 Rodzaje testów

Katalog	Co sprawdza
Data/	Repozytoria na prawdziwym SQLite: sortowania, filtry, ulubione, kaskady, FTS (diakrytyki, prefiksy, AND), przesuwanie znaczników po naprawie, foldery, statystyki.
Services/	<code>OnboardingService</code> (wznawianie, ukończenie, limit odpowiedzi), <code>Settings</code> (domyślne wartości, zepsuty plik, niezany język, kopie obiektów).
Infrastructure/	<code>DisplayHelper.SafeFolderName</code> , <code>Strings</code> (komplet kluczy w czterech językach, zgodność symboli, każdy klucz użyty w źródłach istnieje — test skanuje pliki <code>.cs</code> repozytorium).
Endpoints/	Budowanie zapytań FTS (<code>ToFtsQuery</code>), w tym odporność na operatory FTS.
Workers/	Skaner na tymczasowym folderze z udawanymi plikami: dodawanie, pomijanie świeżo zapisanych, usuwanie, klipy w chmurze, zmiana rozmiaru, brak folderu.
Api/	Cały <code>Program.cs</code> uruchomiony przez <code>WebApplicationFactory<Program></code> (jedna instancja na kolekcję "api"): strona główna i cache-busting, pliki statyczne, konfiguracja folderu, język, motyw, foldery, klipy, segmenty, wyszukiwanie, statystyki, plik <code>.txt</code> (BOM, kolejność, nagłówki), usuwanie, samouczek, endpointy statusu.

7.4 Jak działa uruchomienie aplikacji w teście

`WebApplicationFactory` wykonuje prawdziwy punkt wejścia: `Program.cs` działa od początku do `app.Run()`, przy czym serwer Kestrel jest podmieniony na `TestServer` (bez portu). Dlatego w teście działają też skaner, worker miniatur (bez ffmpeg kończy się kodem 2 dla każdego klipu) i idle watcher (wyłączony zmienną). Wymaga to `public partial class Program` na końcu `Program.cs`.

7.5 Jak dodać test

- Test repozytorium: klasa z konstruktoremwołającym `TestDb.Reset()`; dane przez `TestDb.AddClip(...)` lub bezpośrednio przez repozytoria.
- Test endpointu: `[Collection("api")]`, konstruktor (`ApiFactory factory`) z resetem bazy i ustawień, `factory.CreateClient()`; pomocniki `GetJson`, `PostJson`, `PatchJson`, `PostEmpty`, `Json()`.
- Niewołaj endpointów uruchamiających ffmpeg, Whisper ani Google (wycinanie, transkrypcja zbiorcza, chmura) w ścieżce sukcesu — testuj ścieżki błędów.
- Test powinien paść po zepsuciu kodu: sprawdzono to mutacyjnie (7 celowych błędów w różnych warstwach, każdy wykryty przez co najmniej jeden test).

7.6 Czego testy nie obejmują i jak to sprawdzać ręcznie

Obszar	Jak sprawdzić
ffmpeg: miniatury, wycinanie, naprawa, kopie podglądowe	Własne pliki testowe w osobnym folderze biblioteki; sprawdź wynik w Eksploratorze i w odtwarzaczu (przewijanie, 120+ FPS).

Whisper	<code>KEEPCLIP_WHISPER_MODEL=small</code> dla szybkiej próby; sprawdź dziennik w statusie transkrypcji (silnik, model).
Google Drive	Osobne konto testowe; wysyłka, synchronizacja na drugim komputerze (<code>appProperties</code>), pobranie, odtwarzanie z chmury z przewijaniem. Nie wysyłaj prywatnych klipów użytkownika.
Bufor powtórek	Włącz w ustawieniach, sprawdź status (enkoder, przechwytywanie, audio), zapisz skrótem podczas gry pełnoekranowej i z pulpitu; sprawdź synchronizację audio i długość.
Okno i zasobnik	Zamknięcie z nagrywaniem w tle, drugi start (przywrócenie okna), zmiana monitora, brak WebView2 (tymczasowo zablokuj profil <code>data/webview2</code>).
Interfejs	Tryb serwera w przeglądarce (<code>KEEPCLIP_SERVER_ONLY=1</code> , <code>http://127.0.0.1:8770</code>) i w WebView2; oba motywy; cztery języki; samouczek od zera (usuń klucz <code>onboarding</code> z ustawień).

8. Diagnostyka

Narzędzie	Gdzie	Co daje
Panel logów	przycisk „Logi” (tylko build deweloperski)	Wszystko, co kod pisze na <code>Console.Error</code> i przez <code>DevLog.Add</code> : start bufora, audio, chmura, skany, samouczek.
<code>data/crash.log</code>	obok bazy	Nieobsłużone wyjątki (wątek okna, wątki tła, zadania), błąd <code>WebView2</code> , watchdog powtórek. Plik zaczyna się od nowa po 512 kB. Z datą i wersją aplikacji.
Port serwera	<code>netstat -ano findstr <PID KeepClip.exe></code>	Pozwala odpytać API z przeglądarki lub <code>curl: /api/ping, /api/replay/status, /api/transcribe/status, /api/cloud/status</code> .
Tryb serwera	<code>KEEPCLIP_SERVER_ONLY=1</code>	Stały port 8770, interfejs w zwykłej przeglądarce z narzędziami deweloperskimi.
Status transkrypcji	<code>/api/transcribe/status</code> → <code>log_tail</code>	Dobór modelu, silnik, błędy Whispera.

8.1 Typowe problemy

Objaw	Przyczyna i rozwiązanie
Transkrypcja nie działa, Windows pokazuje „aplikacja częściowo zablokowana”	Smart App Control blokuje niepodpisaną bibliotekę Whispera. Rozwiązanie po stronie autora: podpisywanie plików. Użytkownik może tylko wyłączyć Smart App Control (na stałe).
Bufor powtórek pusty lub „zatrzymał się”	Sprawdź status (<code>error</code> , <code>capture</code>). Zmiana trybu ekranu przez grę restartuje ffmpeg; po 6 awariach bufor się zatrzymuje — zmień ustawienia albo włącz ponownie. W sesji zdalnej lub Sandbox działa tylko GDI.
Nieme powtórki	Log audio: „Bufor audio: szczyt 15s = 0.0000” oznacza brak sygnału z wybranego wyjścia — wybierz inne urządzenie lub „Domyślne” (wszystkie wyjścia).
Proces zabity przy transkrypcji	Brak pamięci GPU. Po restarcie model zostaje zablokowany; można wymusić <code>KEEPCLIP_WHISPER_MODEL</code> .
Okno czarne / komunikat o <code>WebView2</code>	Brak lub uszkodzony runtime <code>WebView2</code> — instalacja ze strony Microsoftu; szczegóły w <code>crash.log</code> .
Klip powyżej 120 FPS zaczyna się	Poczekaj na kopię podglądową (pasek postępu w odtwarzaczu); wymaga ffmpeg.
Skrót powtórki nie działa	Konflikt z inną aplikacją (status <code>hotkey_active=false</code>) — zmień skrót. Klawisz F10 wymaga modyfikatora.
Aplikacja zamyka się sama po kilku minutach	Brak sygnału życia z interfejsu (5 min). Sprawdź, czy strona się ładuje; w trybie serwera ustaw <code>KEEPCLIP_IDLE_EXIT=0</code> .

9. Skalowanie i kierunki rozwoju

KeepClip jest aplikacją jednego użytkownika na jednym komputerze i taka architektura jest celowa: brak serwera, brak kont, brak kosztów. „Skalowanie” oznacza tu cztery różne rzeczy, każda z innym planem.

9.1 Większe biblioteki (tysiące klipów)

- Baza jest gotowa: indeksy po grze i dacie, FTS5 z `bm25`, identyfikatory bez ponownego użycia. Listy mają parametr `limit`; interfejs pobiera 120 klipów i 200 wyników. Przy dużych bibliotekach dodaj stronicowanie (`offset` lub `cursor` po `mtime,id`) w `/api/clips` i `/api/search` oraz doładowywanie przy przewijaniu.
- Skan co 2,5 s przegląda cały folder rekurencyjnie. Dla dziesiątek tysięcy plików zamiast odpytывanie na `FileSystemWatcher` z opóźnionym skanem tylko zmienionych podfolderów; zostaw pełny skan przy starcie i na żądanie.
- Miniatury i kopie podglądowe rosną bez limitu w `data/`. Dodaj sprzątanie kopii podglądowych (LRU po rozmiarze) i pomiar w `/api/stats`.
- Transkrypcja jest sekwencyjna. Whisper na GPU i tak zajmuje całą kartę; dla CPU można równolegle dekodować audio ffmpeg dla następnego klipu (potok), co skraca przerwy między klipami.

9.2 Więcej funkcji bez chaosu

- Serwisy statyczne (`ReplayService`, `OAuthService`, `Transcriber`, `Settings`) utrudniają testy i wymianę implementacji. Kierunek: przenieść je do wstrzykiwania zależności jak `ClipService` i `PlaybackProxyService`, z interfejsami dla ffmpeg (`IMediaTools`), chmury (`ICloudStorage`) i mowy (`ISpeechToText`). Wtedy testy dostaną atrapę, a produkcja prawdziwe implementacje.
- Drugi dostawca chmury (OneDrive, Dropbox, S3) to naturalny pierwszy przypadek dla `ICloudStorage`: `CloudService` używa dziś pięciu operacji (folder, wysyłka, lista, pobranie, koszt) i strumienia z zakresem.
- Podział rozwiązania na projekty: `KeepClip.Core` (Data, Services, Domain, bez WinForms) i `KeepClip.Desktop` (okno, tray, hotkey). Core dałoby się uruchomić także na Linuxie w trybie serwera.
- Migracje bazy jako ponumerowane kroki (tabela `schema_version`) zamiast listy `AddColumn` — dziś wystarcza, przy zmianach typów kolumn przestanie.

9.3 Wiele urządzeń / dostęp z sieci

- Serwer już działa bez okna (tryb serwera), więc pierwszym krokiem jest uwierzytelnianie (token w nagłówku, wydawany lokalnie) i HTTPS, zanim nasłuch wyjdzie poza `127.0.0.1`. Bez tego każdy w sieci mógłby usuwać klipy.
- Synchronizacja biblioteki między komputerami działa dziś przez Dysk Google (`appProperties` odtwarzają grę, czas i długość). Do pełnej synchronizacji (ulubione, foldery, transkrypcje) trzeba eksportować metadane, np. plik JSON w folderze KeepClip na Dysku, scalany przy synchronizacji.
- Transkrypcja na innym komputerze (mocniejsze GPU) to osobny proces roboczy z tym samym API `ISpeechToText` i kolejką zadań; klipy w chmurze już mają wszystko, czego potrzebuje taki worker.

9.4 Jakość i utrzymanie

- CI uruchamia 124 testy przy każdym pushu i przed wydaniem. Kolejne obszary do pokrycia: `CloudService.SyncFromCloudAsync` z atrapą Dysku (logika dopasowania jest złożona), `ReplayService.SaveAsync` z atrapą ffmpeg (wybór segmentów), `Transcriber.SplitOnInternalGaps` (dzielenie po ciszy).
- Podpisywanie kodu w CI usuwa blokady Smart App Control i ostrzeżenia SmartScreen — największy pojedynczy problem użytkowników.
- Dokumentacja żyje w `docs/src/*.html`, a PDF-y powstają poleceniem `node docs/build-pdf.js` (Edge lub Chrome w trybie bez okna). Kod nie ma komentarzy; przy każdej zmianie zachowania zaktualizuj odpowiedni rozdział.
- Telemetrii nie ma i nie powinno być; do diagnozy u użytkowników wystarczy `crash.log` i dziennik statusu transkrypcji.